

BOS DRAM characterization에서 SDPA·MLP 최적화까지

목적

이 문서는 BOS DRAM microbenchmark에서 얻은 memory-parallelism 조건을 실제 Llama 3.2 3B 64K decode의 SDPA, MLP, QKV 및 Wo에 어떻게 적용했는지 발표용 논리와 근거 수치로 정리한다.

핵심 주장은 다음과 같다.

We first characterized the BOS memory subsystem, identified where the model failed to expose the same parallelism, applied workload-compatible corrections, and validated the gains from microbenchmark to kernel to full-model throughput.

Microbenchmark의 96 GB/s는 read-only sequential transport ceiling이다. 실제 kernel이 반드시 이 값을 달성해야 한다는 뜻이 아니다. Microbenchmark는 포화에 필요한 최소 spatial resources, transaction geometry, outstanding depth를 찾는 도구다. 실제 kernel에는 paging, activation multicast, circular-buffer 계약, compute, reduction 및 output 처리가 추가된다.

1. 측정 대상과 용어

- board identity: custom 20-core BOS NPU
- runtime/code architecture: Blackhole
- available worker grid: $5 \times 4 = 20$ cores
- physical DRAM banks: 3
- worker-visible NoC endpoints: bank당 2개, 총 6개
- target workload: Llama 3.2 3B, batch 1, 64K paged-KV decode

Physical bank, worker NoC endpoint, runtime logical DRAM grid, DRAM-interface worker 및 active compute core는 서로 다른 개념이다. **Dram Interface Workers: 6**을 6 physical banks 또는 6 active compute cores로 해석하지 않는다. SDPA stable은 16 active reader/compute cores이고, MLP stable fanout-2는 12 reader/compute workers다.

2. Microbenchmark가 밝힌 포화 조건

2.1 Transaction size와 issue overhead

Single-reader, tagged depth-2, 16 KiB issue batch에서 packet size를 바꿨다.

Packet	Bandwidth
512 B	10.423 GB/s
1 KiB	19.709 GB/s
2 KiB	26.081 GB/s
4 KiB	27.753 GB/s
8 KiB	27.217 GB/s

Packet	Bandwidth
16 KiB	27.035 GB/s

512 B와 1 KiB는 command issue overhead가 크다. Single-reader 조건에서는 4 KiB가 최고였고 8--16 KiB는 추가 이득이 없었다. 그러나 all-bank aggregate에서는 endpoint/service concurrency가 달라져 최적점도 달라졌다.

All-bank, all-6-endpoint에서는 공간 조건을 다음과 같이 고정했다.

- 3 physical banks, bank당 2 endpoints
- endpoint당 reader 1개, 총 6 readers
- endpoint density 1:1:1:1:1:1, bank reader load 2:2:2, NoC reader load 3:3
- endpoint-local DRAM shard, unit-stride access, tagged depth-2

그 위에서 request size 4/8/16 KiB와 tagged batch 32/64 KiB의 3×2 full factorial, 총 6개 permutation을 측정했다. 각 permutation은 profiler 없이 30회 반복했다.

Request	Tagged batch	Requests/batch	Bandwidth
4 KiB	32 KiB	8	92.753 GB/s
8 KiB	32 KiB	4	95.056 GB/s
16 KiB	32 KiB	2	92.675 GB/s
4 KiB	64 KiB	16	92.486 GB/s
8 KiB	64 KiB	8	95.262 GB/s
16 KiB	64 KiB	4	93.701 GB/s

8 KiB는 4 KiB 대비 2.48--3.00%, 16 KiB 대비 1.67--2.57% 높았다. Maximum packet size가 항상 최적은 아니다. 아래 timestamp breakdown은 64 KiB batch의 세 request size만 별도 계측했다. 계측 run은 원인 attribution에만 사용하고 위 profiler-off 30-run 평균을 최종 bandwidth로 사용한다.

Request	Issue	Retire wait	Observed latency mean
4 KiB	26.15%	69.49%	4,310 cycles
8 KiB	13.36%	82.26%	4,652 cycles
16 KiB	6.98%	88.77%	4,968 cycles

Request가 커지면 issue overhead는 감소하지만 service completion latency와 retire wait는 증가했다. All-bank path에서는 8 KiB가 두 비용의 sweet spot이었다.

2.2 Tagged window와 outstanding depth

All-bank, 16 KiB request, depth-2에서 tagged batch를 sweep했다.

Tagged batch	Requests/batch	Bandwidth
16 KiB	1	92.114 GB/s

Tagged batch	Requests/batch	Bandwidth
32 KiB	2	95.792 GB/s
64 KiB	4	95.548 GB/s
128 KiB	8	95.052 GB/s
256 KiB	16	95.528 GB/s

16→32 KiB는 +3.99%였지만 32→256 KiB는 0.78% 범위의 plateau였다. 더 큰 window는 ceiling을 높이지 않았다.

32 KiB batch에서 barrier와 outstanding depth를 비교했다.

Mode	Bandwidth	Depth-2 대비
Tagged depth-2	95.792 GB/s	기준
Tagged depth-3	94.861 GB/s	-0.97%
Full barrier	67.673 GB/s	-29.35%

Depth-2는 full barrier 대비 +41.55%였다. 세 번째 pending slot은 개선하지 않았다. 결론은 maximum queue depth가 아니라 latency를 숨길 수 있는 최소 depth를 유지하는 것이다.

2.3 Reader와 endpoint parallelism

Physical bank0의 두 endpoints에 readers를 균등 배치하고 unit-stride 16 KiB one-packet read를 수행했다.

Total readers	Readers/endpoint	Bandwidth	Retire wait	Observed latency mean
1	1:0	28.209 GB/s	79.09%	2,689 cycles
2	1:1	52.023 GB/s	81.72%	2,949 cycles
4	2:2	51.569 GB/s	90.99%	6,272 cycles
6	3:3	51.378 GB/s	94.03%	9,634 cycles
8	4:4	51.316 GB/s	95.51%	12,943 cycles

1→2 readers는 +84.42%였다. 두 endpoints에 reader 하나씩 배치하면 single bank가 포화됐다. 2→8 readers에서는 bandwidth가 1.36% 감소하고 observed latency는 4.39배, retire wait는 13.79 percentage points 증가했다. 추가 concurrency는 throughput이 아니라 queue/service wait만 늘렸다.

2.4 Bank aggregate scaling

각 physical bank의 two-reader ceiling은 51.666→52.023 GB/s로 균일했다. 그러나 여러 bank를 동시에 활성화하면 독립 bank 합보다 낮았다.

Active banks	Readers	Measured	Independent-bank ideal	Scaling efficiency
bank0+bank1	4	75.055 GB/s	103.689 GB/s	72.39%
bank0+bank2	4	77.704 GB/s	103.990 GB/s	74.72%

Active banks	Readers	Measured	Independent-bank ideal	Scaling efficiency
bank1+bank2	4	74.427 GB/s	103.633 GB/s	71.82%
bank0+bank1+bank2	6	96.139 GB/s	155.656 GB/s	61.76%

Single-bank 평균 51.885 GB/s에서 all-bank 96.139 GB/s로 1.85배 증가했다. Ideal 3배 scaling은 아니었다. 관측은 shared service path backpressure와 일치하지만, 정확한 위치가 aggregate NoC인지 DRAM endpoint/controller arbitration인지는 분리되지 않았다.

2.5 Microbenchmark 결론

1. 모든 physical banks와 endpoints를 사용해야 aggregate roof에 접근한다.
2. Single bank는 endpoint당 reader 하나, 총 2 readers면 포화된다.
3. All-bank request sweet spot은 약 8 KiB다.
4. Tagged window는 32--64 KiB면 충분하다.
5. Pending depth-2는 중요하지만 depth-3는 불필요하다.
6. Plateau 이후 reader, burst 및 queue depth 증가는 latency와 복잡도만 증가시킨다.

3. Vanilla SDPA가 포화 조건에서 벗어난 부분

Vanilla K128 SDPA는 16 readers가 있어도 traffic이 세 endpoints에 집중됐다. Reader 수 자체보다 endpoint service parallelism이 부족했다. Paged KV abstraction 자체가 항상 비연속 주소를 뜻하지는 않는다. 그러나 이 문서의 full-model 및 isolated SDPA runner는 physical blocks를 permutation하여 page table을 만든다. 따라서 page 경계에서 source tile ID가 점프하고, TensorAccessor가 선택하는 endpoint 또는 local address도 달라질 수 있다. 아래 3.1에서 주소 변환과 측정 경로를 분리한다. K128은 64K context에서 chunk-level online softmax update와 merge를 많이 수행했다.

항목	Microbenchmark 포화 조건	Vanilla SDPA
Spatial service	3 banks, 6 endpoints	세 endpoints 집중
Reader balance	endpoint bundle 균형	endpoint별 부하 불균형
Address stream	unit-stride	paged KV translation
Request cadence	pure continuous read	chunk/reducer/compute cadence
Consumer	없음	online softmax와 reducer 존재

3.1 Paged-KV locality 요약

32-token page에서 한 KV head의 K 또는 V payload는 4 tiles = 4.25 KiB다. 현재 shuffled page table은 page 경계마다 physical block을 바꾸고, interleaved layout은 page 내부 tiles도 runtime DRAM views에 분산할 수 있다. 따라서 sharding만으로 큰 request가 생기지 않는다.

V는 page-head sharding 뒤 page 내부 tiles를 직접 burst할 수 있다. K는 DRAM에서 row-major로 읽고 L1의 transposed tile-grid 위치에 배치하므로 contiguous staging read와 L1 stride scatter가 필요하다. 주소 수식, page-size별 payload, interleaved/sharded × sequential/shuffled 실험표와 K staging 설계는 Appendix A에 둔다.

4. SDPA에 적용한 교정과 효과

4.1 Historical stable bundle

초기 stable 이력은 vanilla K128에서 dual-NoC와 6-endpoint distribution을 함께 적용한 bundle 비교다.

Configuration	Effective K/V bandwidth	SDPA latency
Vanilla K128	41.12 GB/s	약 3.47 ms
K128, dual-NoC + 6-endpoint distribution	56.61 GB/s	2.519 ms
변화	+37.67%	약 -27.4%

Stable mapping은 16 active reader/compute cores, endpoint load 3/2/3/3/3/2, NoC load 8/8, 3 physical banks 및 6 worker endpoints를 사용했다. 이 표는 production-oriented bundle 효과를 보여주지만 endpoint 수의 독립 기여를 뜻하지 않는다.

4.2 Controlled K128 endpoint-count A/B

최신 통제 실험은 K128과 dual-NoC를 양쪽에 고정하고 endpoint count만 3→6으로 바꿨다.

Metric	3 endpoints	6 endpoints	Effect
Endpoint load	5/0/5/6/0/0	3/3/2/3/3/2	6개 service point 사용
NoC reader load	5/11	8/8	mapping bundle 내 균형화
Profiled critical span	3.014568 ms	2.408938 ms	-20.09%
Effective K/V bandwidth	47.306 GB/s	59.199 GB/s	+25.14%
Reader K+V barrier mean	1669.096 us	556.299 us	-66.67%
Compute K+V input wait mean	396.584 us	22.817 us	-94.25%

양쪽 PCC는 0.9998791594607118로 같았다. 이 A/B는 6-endpoint를 허용한 현재 mapping 전체가 DRAM service tail과 exposed compute wait를 줄였음을 입증한다. Endpoint 수와 그 결과 생긴 NoC load balance는 분리되지 않았다. dual-NoC는 양쪽 공통이므로 단독 효과도 아니다.

발표에서는 historical +37.67% bundle과 controlled +25.14% endpoint-count A/B를 같은 막대로 합치지 않는다. Microbenchmark의 spatial parallelism 원리가 paged SDPA에서도 유효했다는 인과 근거에는 controlled A/B를 사용한다.

4.3 K chunk 128→256은 별도 algorithmic optimization

6-endpoint 환경에서 K128→K256은 2.51933→2.03641 ms, -19.17%였다. K/V payload byte는 줄지 않는다. Chunk iteration, current-softmax update 및 online merge 횟수가 절반이 된다. Phase decomposition에서 K256-K128 net -150.176 us의 98.84%를 독립 kernel 감소 151.937 us가 설명했다.

Endpoint 분산은 memory parallelism 개선이고 K256은 chunk-level fixed cost 감소다. 발표 그래프에서 두 효과를 같은 색이나 하나의 원인으로 합치지 않는다.

5. Vanilla MLP가 포화 조건에서 벗어난 부분

MLP weight는 static하고 규칙적이므로 KV cache보다 microbenchmark의 contiguous stream에 가깝다. 그러나 vanilla interleaved layout은 tile ownership과 endpoint ownership을 직접 맞추지 못했다. Reader load도 6:5:1로 불균형했다.

항목	Microbenchmark 포화 조건	Vanilla MLP
Weight ownership	endpoint-local shard	interleaved traversal
Endpoint load	균형	6:5:1
Transaction window	32--64 KiB plateau	K-block/CB cadence에 종속
Outstanding depth	depth-2	barrier 또는 제한된 overlap
Consumer	없음	activation multicast와 matmul compute

6. MLP에 적용한 교정과 효과

Stable MLP는 DRAM width-sharded weights, balanced endpoint mapping, 16 KiB read cap, tagged pending depth-2, fanout-2를 사용한다.

A/B	Before	After	Effect
Interleaved→DRAM-sharded MLP	2.229688 ms	1.899062 ms	-14.83%
Endpoint 6:5:1→4:4:4	1.875653 ms	1.472280 ms	-21.51%
Direct weight bandwidth	48.60 GB/s	62.93 GB/s	+29.47%

실제 stable request는 W1/W3 BFP4가 22 tiles × 576 B = 12,672 B, W2 BFP8이 8 tiles × 1,088 B = 8,704 B다. 둘 다 16 KiB cap 이하이며 microbenchmark의 8--16 KiB plateau 범위에 있다. Application의 tile/block geometry 때문에 8 KiB를 기계적으로 강제하지 않았다.

Fanout-2의 12 readers를 “DRAM 포화에 12 readers가 필요했다”고 설명하면 틀린다. Synthetic transport는 all-bank 6 readers에서 이미 포화됐다. Fanout-2는 6 interface-worker groups의 weight를 12 compute partitions에 공급하여 compute parallelism과 endpoint ownership을 맞추는 구조다.

6.1 발표 범위

발표에는 A/B가 끝난 DRAM-sharded weights, endpoint 4:4:4 balance, fanout-2와 full-model waterfall만 넣는다. Application effective weight bandwidth 약 60--63 GB/s는 개선됐지만 read-only synthetic roof 약 95--96 GB/s를 포화했다고 주장하지 않는다. Reader/compute decoupling과 추가 pipeline 계측은 Appendix B의 후속 과제로 둔다.

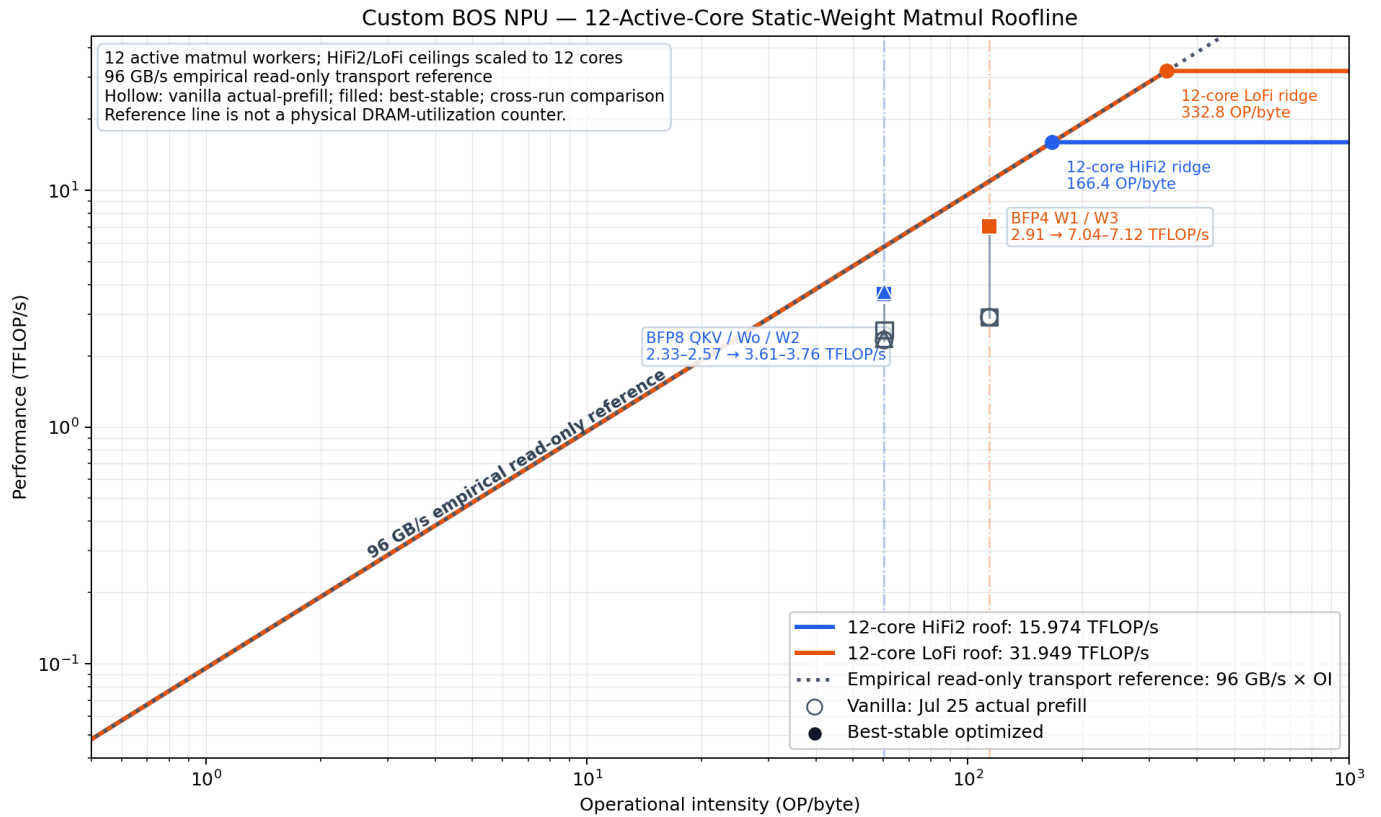
7. QKV와 Wo에 같은 원리를 확장

QKV와 Wo도 static-weight decode matmul이다. MLP에서 검증한 DRAM-sharded balanced fanout-2를 적용했다.

Op	Interleaved latency	DRAM-sharded latency	Kernel change	Effective weight BW
QKV	433.448 us	278.943 us	-35.64%	38.55→59.91 GB/s
Wo	236.042 us	165.835 us	-29.74%	42.48→60.46 GB/s

Grouped concat은 DRAM bandwidth 최적화가 아니다. SDPA output의 generic layout/data-movement pipeline을 줄이고 Wo가 소비할 flattened L1 shard를 12 cores에서 직접 만든다. Exact grouped A/B의 layer mean은 4.469103→4.352791 ms, -2.60%였다.

7.1 Static-weight matmul roofline reference



Hollow points는 vanilla actual-prefill profile, filled points는 best-stable profile이다. 점 사이 이동은 동일한 padded issued-operation 정의에서 measured TFLOP/s가 증가한 방향을 보여준다. 96 GB/s × OI 점선은 empirical read-only transport reference이며 physical DRAM-utilization counter가 아니다. Vanilla와 stable은 cross-run profile이므로 최종 개선폭은 Section 10의 same-session waterfall을 사용한다.

8. Microbenchmark와 application 효과가 다른 이유

Microbenchmark	SDPA/MLP application
Pure read-only	compute와 output 처리 포함
Unit-stride contiguous	paged KV 또는 K-block traversal
지속적인 request issue	chunk/K-block phase cadence
Consumer 없음	CB reserve/push/wait/pop 계약
Activation 없음	activation multicast 필요
Reduction 없음	SDPA online softmax/reducer 존재
96.139 GB/s transport roof	stable kernels 약 60--70 GB/s

Stable W1/W3/W2의 application-level effective weight bandwidth는 measured read-only transport roof 수치의 약 64--65%에 해당한다. 이는 hardware counter 기반 DRAM 또는 memory-bus utilization이 아니다. Consumer

input wait는 kernel 시간의 약 67%지만 pending request는 projection 내부에서 대부분 유지됐다. 남은 gap은 단순 reader 부족이 아니라 DRAM service completion, CB publish/consume cadence, activation multicast 및 compute phase가 섞인 결과다.

SDPA도 약 70 GB/s로 synthetic ceiling보다 낮다. Paged addressing, online softmax, reducer 및 K/V compute가 없는 read-only benchmark와 같은 수치를 기대할 수 없다.

8.1 Metric boundary 요약

Microbenchmark GB/s, SDPA effective K/V GB/s, matmul effective weight GB/s와 full-model tok/s는 정의가 다르다. 본문에서는 같은 configuration의 A/B와 end-to-end waterfall만 인과 근거로 사용한다. Roof 대비 비율은 **application-level effective bandwidth relative to the measured read-only transport roof** 라고만 표현하며 DRAM utilization으로 부르지 않는다. 세부 정의와 금지 비교는 Appendix C에 둔다.

9. 제외한 변경

Tagged depth-3, cross-chunk prefetch, directed-link route 변경, DRAM-nearest placement는 개선이 없거나 regression을 보여 stable에서 제외했다. Reduce-only helper는 deadlock 이력 때문에 금지했다. 전체 수치와 판정은 Appendix D에 둔다.

10. Full-model 검증

같은 source/build와 host session에서 synthetic zero 64K KV, profiler off, warmup 3 tokens 뒤 50-token window를 구성별 5회 측정했다. $\pm$ 는 sample standard deviation이다.

단계	Throughput mean $\pm$ SD	CV	직전 대비	Vanilla 대비
Vanilla-equivalent K128	5.122967 $\pm$ 0.003086 tok/s	0.0602%	기준	기준
+ SDPA K256/6EP bundle	6.415537 $\pm$ 0.004162 tok/s	0.0649%	+25.23%	+25.23%
+ DRAM-sharded MLP	7.637903 $\pm$ 0.000543 tok/s	0.0071%	+19.05%	+49.09%
+ grouped concat, QKV/Wo	8.204496 $\pm$ 0.003359 tok/s	0.0409%	+7.42%	+60.15%

모든 CV는 0.065% 이하이고 인접 단계 95% CI는 겹치지 않았다. 별도 single-run waterfall의 **5.124290→8.207842 tok/s, +60.18%**도 반복 평균과 일치했다. Microbenchmark에서 얻은 방향이 isolated kernel 개선뿐 아니라 full-model critical path 개선으로 이어졌다. 단, 이 결과는 actual prefill이 아니라 zero-initialized synthetic paged KV decode다.

11. 발표 구성 권고

Slide 1: Finding the minimum resources required to saturate DRAM

- 1→2 readers: 28.209→52.023 GB/s, +84.42%
- 2→8 readers: bandwidth 정체, latency 2,949→12,943 cycles
- all-bank/all-6-endpoint: 96.139 GB/s
- 결론: maximum concurrency가 아니라 minimum saturation point

Slide 2: Why vanilla kernels fell below the roof

- SDPA: endpoint concentration, paged KV, chunk/reducer cadence
- MLP: interleaved static weights, 6:5:1 endpoint imbalance
- Microbenchmark 조건과 application 제약을 좌우 비교

Slide 3: Workload-compatible corrections

- SDPA endpoint distribution과 K256을 원인별로 분리
- MLP DRAM sharding, balance, request geometry, depth-2; 검증된 stable 변경만 제시
- MLP의 약 60--63 GB/s와 남은 reader/compute decoupling은 후속 과제로 분리
- QKV/Wo static-weight 확장

Slide 4: Synthetic Transport Roof vs. Application Data Delivery

- Read-only transport microbenchmark: 약 95--96 GB/s
- SDPA / static-weight matmul: 약 60--70 GB/s effective application rate
- Different denominators; not a physical DRAM utilization comparison
- Remaining difference includes paging, CB synchronization, activation delivery, compute, and reduction

Slide 5: End-to-end waterfall

- 5.123→6.416→7.638→8.204 tok/s, 각 5-run mean
- 최종 +60.15%, 모든 CV $\leq 0.065\%$
- footnote: synthetic zero 64K KV, profiler off, vanilla-equivalent baseline

12. 사용 가능한 주장과 피해야 할 주장

사용 가능:

The microbenchmark identified the minimum spatial resources and transaction window required to approach the measured memory roof.

Applying workload-compatible parts improved SDPA and static-weight matmul bandwidth, but application kernels retained compute and synchronization constraints absent from the synthetic benchmark.

The final configuration improved synthetic-zero-KV 64K decode throughput by 60.15% over the vanilla-equivalent baseline (five-run mean; all CVs at or below 0.065%).

피해야 함:

All kernels saturate DRAM.

Dual NoC doubled bandwidth.

Twelve MLP readers were required to saturate DRAM.

The 96.139 GB/s microbenchmark roof is the hardware specification.

Appendix A. SDPA paged-KV 주소와 burst 설계

Paged KV cache의 논리 순서는 token 0, 1, 2, ...로 연속이다. 저장은 고정 크기 physical block pool을 사용한다. 현재 Python cache shape는 다음과 같다.

```
[physical_block, kv_head, tokens_in_block, head_dim]
```

Page table은 logical block 번호를 physical block 번호로 바꾼다. B 를 block당 token 수라 하면 논리 token t 의 block은 다음처럼 정해진다.

```
logical_block = floor(t / B)
token_in_block = t % B
physical_block = page_table[logical_block]
```

TT tile은 sequence 방향 32 tokens를 묶는다. Kernel은 token 대신 sequence tile row s 를 사용한다. $block_size_t = B / 32$, head-dimension tile 수를 Wt 라 하면 실제 source tile row의 시작 ID는 코드상 다음 수식이다.

```
virtual_block    = floor(s / block_size_t)
physical_block   = page_table[virtual_block]
row_in_block     = s % block_size_t
block_stride     = num_kv_heads * block_size_t * Wt
head_offset      = kv_head * block_size_t * Wt
physical_tile_id = physical_block * block_stride
                  + head_offset
                  + row_in_block * Wt
```

이 수식은 `dataflow_common.hpp::virtual_seq_tile_id_to_physical_tile_id()`에 그대로 구현돼 있다. Reader는 K와 V의 각 sequence tile row마다 이 함수를 호출한 뒤 `noc_async_read_tile()`을 issue한다.

Page 내부와 page 경계

같은 page 안에서는 `physical_block`과 `head_offset`이 고정되고 `row_in_block`만 증가한다. 따라서 한 head의 source tile rows는 page 내부에서 규칙적이다. Head dimension 128이면 $Wt=4$ 이고, 같은 row의 네 column tile은 `physical_tile_id, +1, +2, +3`이다. 다음 sequence row 시작은 $+4$ 다.

Page 경계에서는 `row_in_block`이 0으로 돌아가고 `physical_block`을 다시 page table에서 읽는다. 예를 들어 page table 앞부분이 `[17, 4, 29, 8]`이면 논리 page 순서는 연속이어도 source는 physical block $17 \rightarrow 4 \rightarrow 29 \rightarrow 8$ 로 이동한다. 이때 다음 tile ID가 이전 tile ID 바로 뒤라는 보장이 없다. TensorAccessor가 새 physical tile ID를 NoC 주소로 바꾸므로 DRAM endpoint와 local address도 바뀔 수 있다. 한 page에서 끝난 read를 다음 page까지 하나의 unit-stride burst로 단순 연장할 수 없다.

반대로 page table이 `[0, 1, 2, 3, ...]`이고 allocator layout도 연속이라면 page 경계에서도 연속성이 유지될 수 있다. 따라서 정확한 표현은 다음과 같다.

Paged attention preserves locality within a physical page, but does not guarantee locality across page boundaries; cross-page locality is determined by the page table and allocator placement.

현재 측정에서 실제로 비연속인 이유

이 문서의 64K full-model runner는 `PAGE_BLOCK_SIZE=32`를 사용한다. `create_tt_page_table()`은 `torch.randperm(max_num_blocks)`과 그 inverse permutation으로 logical→physical mapping을 만든다. 32-token page는 sequence tile row 하나와 같아 `block_size_t=1`이다. 따라서 인접한 sequence tile row마다 page-table lookup과 임의 physical-block 전환이 발생한다. 이는 paged API의 가능성만이 아니라 해당 benchmark의 실제 조건이다.

Controlled isolated SDPA runner도 shuffled page table을 사용한다. 다만 그 실험의 page block이 128 tokens면 `block_size_t=4`이므로 네 sequence tile rows 동안은 같은 physical block에 머물고 다섯 번째 row에서 점프한다. 즉 page 크기가 커지면 cross-page jump 빈도는 줄지만, page 내부 head layout, K의 transposed L1 destination, CB cadence와 online-softmax 비용은 그대로 남는다.

32-token page가 제공하는 실제 연속 구간

현재 performance KV dtype은 BFP8_B이고 raw tile은 1,088 B다. Head dimension 128은 tile width 네 개이므로 한 sequence tile row, 즉 32 tokens × 한 KV head × K 또는 V 하나는 다음 크기다.

$$4 \text{ tiles} \times 1,088 \text{ B} = 4,352 \text{ B} = 4.25 \text{ KiB}$$

Page 크기를 늘렸을 때 한 head의 한 K 또는 V tensor 안에서 page 경계 전까지 이어지는 logical source-tile payload는 다음과 같다.

Page tokens	Sequence tile rows	Tiles/head/page	K 또는 V/head/page	K 또는 V/8 heads/page	K+V/8 heads/page
32	1	4	4.25 KiB	34 KiB	68 KiB
64	2	8	8.5 KiB	68 KiB	136 KiB
128	4	16	17 KiB	136 KiB	272 KiB
256	8	32	34 KiB	272 KiB	544 KiB

SDPA reader는 보통 한 KV head의 K와 V를 별도 phase와 별도 tensor에서 읽는다. 따라서 32-token page의 reader 관점 연속 구간은 68 KiB가 아니라 K 또는 V 각각 4.25 KiB다. 위 표도 logical tile-ID run을 뜻한다. Interleaved TensorAccessor가 이를 하나의 physical DRAM byte run으로 유지하는지는 별도 문제다.

Page 확대가 자동으로 큰 request를 만들지는 않는다

Stable generic paged reader는 sequence row와 head-dimension column을 순회하며 tile마다 `noc_async_read_tile()`을 호출한다. BFP8에서는 개별 issue가 1,088 B다. Page를 32→64/128로 바꾸면 page-table lookup과 physical-block jump 빈도는 각각 1/2, 1/4로 줄지만, reader를 바꾸지 않으면 NoC request 수는 그대로다.

Page-head ND-sharded experimental V path는 page 경계를 넘지 않는 범위에서 최대 8 tiles를 `noc_async_read()` 한 건으로 합친다. 최대 payload는 다음과 같다.

$$8 \text{ tiles} \times 1,088 \text{ B} = 8,704 \text{ B} = 8.5 \text{ KiB}$$

이는 all-bank microbenchmark의 약 8 KiB request sweet spot과 가깝다. 그래서 현재 가장 강한 가설은 **64-token page + page-head sharding + 8-tile V coalescing**이다. 64-token page의 한 head subpage가 정확히 8 tiles라 한 request로 처리할 수 있고, 32-token 대비 page transition도 절반이다.

K는 source의 같은 row 네 tiles가 연속이어도 L1 destination을 transpose 형태로 채워야 해 destination pointer가 strided다. 현재 K path가 tile별 read를 유지하는 이유다. 따라서 page 확대와 sharding의 이득은 먼저 V에서 크게 나타날 가능성이 높고, K에는 scatter-capable read 또는 별도 transpose staging이 필요할 수 있다.

K 연속 read와 L1 stride scatter 설계

현재 cache K는 DRAM에 transpose 형태로 저장되지 않는다. Model은 **transpose_k_heads=False**를 사용하고 K와 V를 모두 **[physical block, KV head, token, head_dim]** 순서로 저장한다. Stable reader는 같은 sequence row의 네 BFP8 tiles를 **noc_async_read_tile()** 네 건으로 읽되, destination을 **col * chunk_rows + row**만큼 띄워 K compute CB의 transposed tile-grid 순서를 만든다. 즉 transpose는 DRAM layout이 아니라 reader의 L1 destination 배치에서 발생한다.

NoC의 단일 contiguous read는 한 source run을 한 destination run으로 복사한다. 연속 source 네 tiles를 transposed CB의 strided destinations로 직접 scatter할 수 없다. 따라서 K를 coalesce하려면 다음 staging이 필요하다.

```
page-head DRAM shard: K00 K01 K02 K03
                    4,352-B contiguous read
                    ↓
row-major L1 staging: K00 K01 K02 K03
                    ↓ local-L1/NoC strided tile copies
transposed compute CB: K00 ... K01 ... K02 ... K03
```

32-token page에서 staging buffer는 core당 **4 × 1,088 B = 4.25 KiB**, double buffer는 **8.5 KiB**다. 값이나 산술 순서는 바뀌지 않고 tile 위치만 순열 변경하므로 dataflow contract가 맞으면 exactness를 유지할 수 있다. 다음 page burst를 issue하는 동안 현재 staging slot을 scatter하도록 double buffering해야 DRAM 절감이 L1 reorder 대기로 치환되지 않는다.

동일한 16 KiB tagged batch의 single-reader synthetic sweep은 **1 KiB 19.709, 2 KiB 26.081, 4 KiB 27.753 GB/s**였다. 이는 1→4 KiB request에서 bandwidth **+40.8%**인 transport 상한 사례다. 별도의 SDPA-shaped all-endpoint reference는 **1,088 B packet에서 53.60 GB/s**, 4 KiB packet에서 **66.50 GB/s**였지만 두 행의 burst geometry가 완전히 같지 않아 exact request-size A/B로 취급하지 않는다. 현재의 보수적 가설은 K-read phase 약 **+20--25%**, K와 V를 합친 memory phase 약 **+10%**, 전체 SDPA 약 **+3--8%**다. L1 scatter, CB synchronization 및 compute overlap이 이 범위를 낮출 수 있다.

필수 A/B는 다음과 같다.

변수	값
DRAM layout	interleaved / page-head sharded
K request	1 / 2 / 4 BFP8 tiles (1,088/2,176/4,352 B)
Page table	sequential / shuffled
K destination	direct tile scatter / contiguous staging + L1 scatter

변수	값
관측값	K-read cycles, issue count, barrier/retire wait, L1-reorder cycles, SDPA latency, PCC

page-head sharding + 4-tile K read는 32-token page 내부에서만 burst한다. Shuffled page table이어도 page 내부 4.25 KiB run은 보존할 수 있지만 page 경계를 넘는 coalescing은 하지 않는다. V는 같은 layout에서 destination도 row-major 연속이므로 staging 없이 직접 4/8-tile burst가 가능하다.

메모리 측 최적화 종료 판정

KV sharding은 SDPA 전용이다. MLP에는 KV가 없으며 별도의 static-weight DRAM sharding, fanout-2 reader, 12 compute workers, endpoint/NoC balance와 W2 block-width 교정을 사용한다. 따라서 K staging을 구현했다고 SDPA와 MLP 커널 전체 최적화가 자동으로 끝나는 것은 아니다.

다만 다음 조건을 모두 만족하면 memory-side layout/request optimization은 사실상 종료 후보로 분류할 수 있다.

1. SDPA의 page-head sharded K/V가 page 내부 4--8 KiB request를 만들고 shuffled page table에서도 정확성을 유지한다.
2. K staging의 L1 scatter가 DRAM issue 절감을 상쇄하지 않으며 K/V wait와 endpoint finish skew가 감소한다.
3. MLP의 sharded weight reader가 workload-matched roofline 대비 충분한 bandwidth에 도달하고 input-CB starvation이 더 이상 지배적이지 않다.
4. Request size, tagged depth, reader 수를 더 늘려도 end-to-end latency가 plateau를 보인다.

이후 남은 병목은 memory placement보다 matmul utilization, online-softmax/reduction cadence, CB synchronization, operator gap과 host/CQ overlap으로 분류한다. 즉 이 작업은 DRAM 최적화의 마지막 큰 후보지만 전체 kernel optimization의 종료 선언은 측정 뒤에만 가능하다.

Generic height-sharded KV는 과거 약 24 GB/s대로 악화됐다. 이는 (physical page, KV head)를 하나의 subpage shard로 정의하는 page-head sharding과 다른 layout이다. Generic failure를 page-head 방식의 실패 근거로 쓰지 않지만, "sharding이면 자동으로 빨라진다"는 주장도 하지 않는다.

Page 확대의 비용도 있다. Multi-user 동적 allocator에서는 마지막 page의 internal fragmentation이 user당 최대 page_size-1 tokens까지 증가하고, 작은 sequence 사이의 block 재사용 유연성이 낮아진다. Batch 1의 고정 64K context는 page size로 나누어떨어지므로 이 비용이 작지만 deployment 일반값을 결정하려면 workload별 검증이 필요하다.

Interleaved와 sharded를 별도로 봐야 하는 이유

Page-table discontinuity와 DRAM memory layout은 다른 층이다. Page table은 어떤 physical block을 읽을지 정한다. Interleaved/sharded TensorAccessor는 선택된 physical tile ID를 어느 DRAM view와 local address로 보낼지 정한다. 따라서 다음 네 경우가 모두 가능하다.

Page table	DRAM layout	예상 특성
sequential	interleaved	logical pages는 순차지만 tiles가 views에 분산될 수 있음
shuffled	interleaved	page jump와 view 분산이 함께 존재
sequential	sharded	allocator와 shard ownership이 맞으면 긴 local run 가능
shuffled	sharded	page 내부 local run은 가능하지만 page 사이 shard/address jump 가능

Sharding만 켜다고 shuffled page table이 정렬되는 것은 아니다. 반대로 page table만 sequential하게 만들어도 interleaved endpoint striping은 남을 수 있다. 그래서 후속 실험은 **page-table order × memory layout**의 **2×2** factorial이 가장 명확하다.

Page-table order	Memory layout	목적
sequential	interleaved	interleaved 자체 비용 기준
shuffled	interleaved	page permutation 비용
sequential	sharded	sharding의 최대 locality 이득
shuffled	sharded	실제 paged 조건에서 sharding 잔여 이득

현재 결과로 확정 가능한 것은 shuffled page-table runner가 page 경계 source continuity를 깨뜨린다는 사실이다. DRAM row-buffer hit rate와 physical burst 길이는 counter 또는 address trace 없이 확정하지 않는다.

이 차이를 “SDPA가 memory rule을 위반했다”고 단정하지 않는다. 정확한 표현은 다음과 같다.

The vanilla SDPA kernel did not expose the endpoint-level parallelism observed in the saturation benchmark.

Appendix B. MLP 후속 pipeline 조사

이번 발표에는 위에서 A/B가 완료된 변경만 포함한다.

- Interleaved→DRAM-sharded static weights: latency **-14.83%**
- Endpoint reader balance **6:5:1→4:4:4**: latency **-21.51%**
- Direct effective weight bandwidth **48.60→62.93 GB/s**: **+29.47%**
- Fanout-2와 12 compute partitions: 6 interface-worker groups와 compute ownership을 맞춘 stable 구성
- Full-model waterfall의 **+DRAM-sharded MLP** 단계: 직전 구성 대비 throughput **+19.05%**

발표에서는 이를 “MLP가 DRAM을 포화했다”거나 “12 readers가 transport 포화에 필요했다”고 표현하지 않는다. Application effective weight bandwidth 약 **60--63 GB/s**는 read-only synthetic roof 약 **95--96 GB/s**보다 낮고, 양쪽 metric도 consumer가 없는 transport와 matmul 전체 duration이라는 차이가 있다. 안전한 결론은 sharding과 endpoint ownership 교정이 measured MLP latency를 줄였지만 memory delivery와 consumer pipeline 사이에 추가 headroom이 남는다는 것이다.

추가 최적화는 발표 이후 후속 조사로 미룬다. 우선순위는 다음과 같다.

1. 동일 shard와 주소에서 reader-only kernel과 full matmul을 A/B하여 pure delivery와 consumer backpressure를 분리한다.
2. **weight issue, retire wait, CB reserve wait, compute input wait, pack/write** cycles를 별도 계측한다.
3. 현재 block을 계산하는 동안 다음 weight block을 issue하는 2/3-block staging ring을 검토한다.
4. 실제 weight request가 endpoint-local **4--8 KiB** contiguous packet인지 주소와 request count로 확인한다.
5. Request 확대보다 CB starvation과 math-engine utilization이 먼저 개선되는지 검증한다.

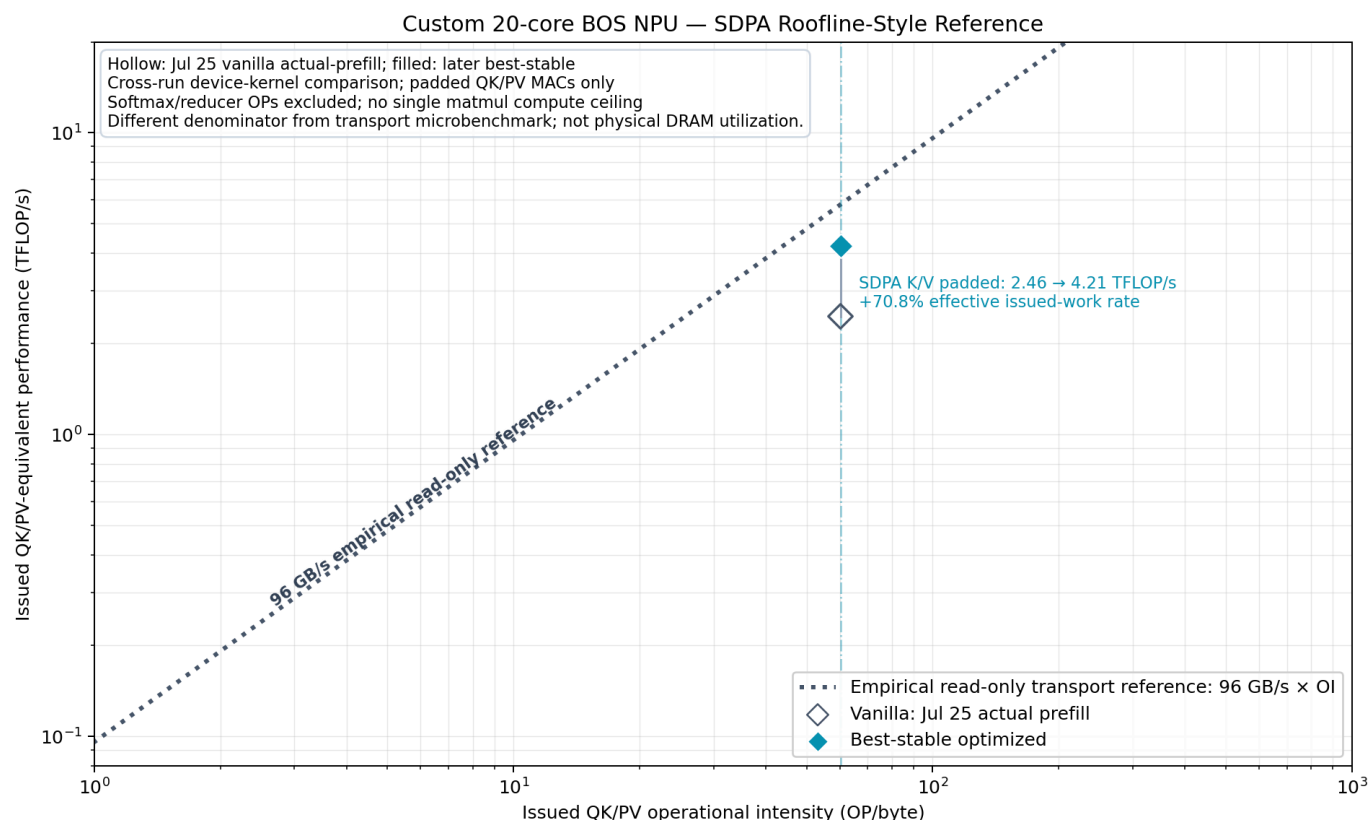
Reader-only bandwidth가 높고 full matmul만 낮으면 compute/CB backpressure가 주원인이다. Reader-only도 낮으면 request geometry, shard ownership 또는 endpoint service가 남은 원인이다. 이 attribution 전에는 fused kernel, 더 큰 pool 또는 reader 증설을 stable로 승격하지 않는다.

Appendix C. Metric 정의와 비교 경계

Metric	분자/분모	포함 범위	사용 목적
Microbenchmark GB/s	실제 read payload / kernel time	read-only transport	measured memory roof
SDPA effective K/V GB/s	logical K+V payload / critical span	paging, compute, reduction 포함	같은 SDPA A/B
Matmul effective weight GB/s	logical weight bytes / op duration	activation, CB, compute 포함	같은 projection A/B
Device FW duration 합	profiler op durations 합	overlap 시 wall time과 다름	layer attribution
Full-model tok/s	measured decode tokens / wall time	28 layers, profiler off	end-to-end 결과

서로 다른 metric의 GB/s를 같은 물리 counter처럼 빼거나 나누지 않는다. Microbenchmark 96.139 GB/s와 SDPA 59--70 GB/s의 비율은 roof gap을 설명하는 참고값이지 DRAM utilization counter가 아니다.

SDPA roofline-style reference



SDPA는 QK/PV matmul, paging, online softmax, reducer와 CB synchronization이 섞인 composite kernel이다. 그래프의 세로축은 padded issued QK/PV-equivalent rate이고 점선은 read-only transport reference다. 따라서 점과 점선의 간격을 physical DRAM utilization로 읽지 않는다. 이 그래프는 vanilla→stable의 issued-work rate 이동만 보여주는 roofline-style reference다.

Appendix D. Negative controls

최적화는 최대값을 무조건 선택하지 않았다.

변경	관측	판정
Single-bank readers 2→8	BW -1.36%, latency 4.39×	2 readers에서 중단
Aggregate tagged depth 2→3	-0.97%	depth-2 유지
MLP tagged depth-3	latency +0.224%	제외
SDPA tagged cross-chunk prefetch	56.500→56.374 GB/s	제외
SDPA directed-link overlap 최소화	wall +0.362%	제외
MLP DRAM-nearest placement	유의미한 개선 없음	제외
SDPA reduce-only helper	deadlock 이력	금지

이 negative controls가 “무작위로 모든 optimization을 컷다”는 해석을 막는다. Plateau 이전까지만 parallelism을 늘리고 plateau 이후 복잡도는 제거했다.

Appendix E. 관측, 추론, 미검증 가설

관측 사실

- Single bank는 two readers/two endpoints에서 약 52 GB/s plateau에 도달했다.
- All-bank/all-6-endpoint read-only ceiling은 약 96 GB/s였다.
- SDPA 6-endpoint bundle, MLP DRAM sharding/balance, QKV/Wo sharding은 각각 measured latency를 줄였다.
- Five-run full-model mean은 5.122967→8.204496 tok/s, +60.15%였고 모든 CV는 0.065% 이하였다.

근거가 강한 추론

- Vanilla SDPA의 endpoint concentration은 spatial service parallelism을 제한했다.
- Static MLP/QKV/Wo weights는 DRAM width sharding과 endpoint ownership 정렬에 적합하다.
- Plateau 이후 reader/depth 증가는 유효 latency hiding보다 queueing 비용을 더 늘린다.

미검증 가설

- Multi-bank scaling efficiency 61.76%를 형성하는 정확한 shared bottleneck 위치.
- Application의 남은 27--38% roof gap에서 DRAM controller, NoC arbitration, CB cadence 및 compute가 각각 차지하는 비율.
- Page-aware KV placement가 paged SDPA의 unit-stride locality를 얼마나 회복할 수 있는지.

Appendix F. 근거 문서와 artifact

- [Single-bank/all-bank DRAM saturation](#)
- [Stable roofline justification](#)
- [Presentation-safe static-weight matmul roofline \(SVG\)](#)
- [Presentation-safe SDPA roofline-style reference \(SVG\)](#)
- Roofline generator: [../benchmark-results/assets/2026-08-17-bos-roofline-presentation-safe.py](#)

- Paged tile mapping: `/home/iris_hb4/tt-metal-hb4/ttnn/cpp/ttnn/operations/transformer/sdpa_decode/device/kernels/dataflow/dataflow_common.hpp`
- Paged K/V reader: `/home/iris_hb4/tt-metal-hb4/ttnn/cpp/ttnn/operations/transformer/sdpa_decode/device/kernels/dataflow/reader_decode_all.cpp`
- 64K page size: `/home/iris_hb4/tt-metal-hb4/models/bos_model/llama32/tests/benchmark_llama32_3b_64k_decode.py`
- Random page mapping: `/home/iris_hb4/tt-metal-hb4/models/bos_model/llama32/run_llama32.py`
- SDPA 3EP vs 6EP metrics
- SDPA K-chunk fixed-cost decomposition
- MLP fanout-2 tagged two-block
- Attention QKV/Wo DRAM-sharded A/B
- 64K optimization waterfall
- DRAM timestamp logs:
`/home/iris_hb4/benchmark_runs/dram_bank0_reader_retire_wait_2026_08_16_09_00_00`
- Full-model 5-run logs:
`/home/iris_hb4/benchmark_runs/llama32_3b_64k_waterfall_repeats5_2026_08_09_17_20_12`

Appendix G. 한계

- 96.139 GB/s는 이론 hardware spec이 아니라 measured read-only transport roof다.
- Full-model waterfall은 actual-prefill이 아니라 synthetic zero KV다.
- Final stable은 bitwise exact가 아니다. Fixed-input full logits PCC는 약 0.9993이고 top-1은 동일했다.
- `dual-NoC`의 단독 기여는 분리되지 않았다.
- Timestamp latency는 pure DRAM CAS latency가 아니라 issue-to-retire observed completion latency다.
- Full-model runner의 random page permutation은 해당 측정의 실제 조건이지만 production allocator의 평균 fragmentation을 측정한 값은 아니다. Sequential/fragmented page-table A/B 전에는 일반 deployment 전체로 정량 비율을 확대하지 않는다.

Appendix H. PPT 제작 전 체크리스트

- 첫 topology slide에 `custom 20-core BOS NPU`, Blackhole runtime, `3 banks/6 endpoints`를 함께 쓴다.
- `96.139 GB/s`에는 `measured read-only transport roof`, hardware specification 아님을 붙인다.
- Microbenchmark GB/s, SDPA effective K/V GB/s, matmul effective weight GB/s를 같은 축에 놓지 않는다.
- SDPA historical bundle과 controlled endpoint-count A/B를 분리한다.
- Full-model 막대에는 5-run error bar와 `synthetic zero KV, vanilla-equivalent` footnote를 붙인다.
- 정확성은 `PCC≈0.9993, same top-1/top-5, not bit-exact`로 쓴다.
- 관측 사실, 강한 추론, 미검증 bottleneck을 색이나 선 종류로 구분한다.
- Raw logs와 source report 경로는 appendix에 둔다.

이 체크리스트를 지키면 현재 문서만으로 architecture characterization→kernel correction→full-model validation의 발표 흐름을 구성할 수 있다. 남은 검증은 현재 결과의 유효성을 깨지 않으며 원인과 ceiling 위치를 더 좁히는 작업이다.

Appendix I. Open evidence backlog

P0: DRAM sharded vs interleaved 통제 A/B

현재 MLP에서는 interleaved→DRAM-sharded가 2.229688→1.899062 ms, -14.83%였지만 microbenchmark에서 layout 자체만 분리하지 않았다. 따라서 row-buffer locality, physical-address continuity 또는 endpoint ownership 중 무엇이 개선을 만들었는지는 미확정이다.

- ☐ 동일 payload와 working set을 사용한다.
- ☐ reader 6개, endpoint density 1:1:1:1:1:1, bank load 2:2:2, NoC load 3:3을 고정한다.
- ☐ request 8 KiB, tagged batch 32/64 KiB, depth-2를 고정한다.
- ☐ logical access order와 reader coordinates를 고정하고 memory layout만 바꾼다.
- ☐ 각 configuration을 profiler 없이 30회 반복한다.
- ☐ aggregate bandwidth, endpoint별 completion latency, retire wait, finish skew를 기록한다.
- ☐ physical-address continuity, endpoint transition count 및 request fragmentation을 정적 검증한다.
- ☐ 이후 동일 MLP matmul에서 memory config만 바꾼 application A/B를 수행한다.
- ☐ Paged-KV 확장은 page-table sequential/shuffled × interleaved/sharded 2×2로 측정한다.
- ☐ 각 2×2 cell에서 page-boundary endpoint transition과 physical tile-ID delta를 기록한다.
- ☐ Shuffled page table에서 page size 32/64/128/256을 sweep한다.
- ☐ 각 page size에서 V coalescing off/on을 분리한다.
- ☐ 우선 32 vs 64, interleaved vs page-head sharded를 최소 A/B로 실행한다.
- ☐ K와 V bandwidth/barrier를 따로 기록하여 transpose destination 영향을 분리한다.

Static-weight layout A/B와 paged-KV 2×2는 다른 실험이다. 전자는 layout 자체를 분리하고, 후자는 page-table permutation과 layout의 interaction을 분리한다.

통과 전 사용 가능한 주장은 DRAM sharding improved measured MLP latency by 14.83%다. Sharding improved row-buffer hit rate 또는 Sharding created larger physical bursts는 사용하지 않는다.

P1: 8 KiB optimum의 독립 검증

- ☐ All-bank/all-6-endpoint에서 8 KiB request × depth 1/2/3을 측정한다.
- ☐ 8 KiB request × tagged batch 16/32/64/128 KiB를 측정한다.
- ☐ profiler-off throughput과 별도 timestamp attribution을 분리한다.
- ☐ reader launch offset을 sweep하여 동시 endpoint arbitration 영향을 확인한다.

현재 8 KiB request의 sweet spot은 4/8/16 KiB × 32/64 KiB batch에서 재현됐다. 그러나 depth 비교는 16 KiB request 중심이므로 8 KiB + depth-2 전체 조합의 독립 최적성은 아직 미검증이다.

P2: Aggregate ceiling 위치

- ☐ endpoint/DRAM/fabric counter가 있으면 arbitration stall과 queue occupancy를 수집한다.
- ☐ counter가 없으면 endpoint별 launch offset, latency p50/p95 및 finish skew로 범위를 좁힌다.
- ☐ 6-reader와 12-reader의 동일-condition latency/throughput을 다시 확인한다.

현재 확정 가능한 것은 balanced 6-reader injection에서 약 95--96 GB/s plateau가 형성된다는 사실뿐이다. 정확한 ceiling 위치가 aggregate NoC인지 endpoint/DRAM service인지 확정하지 않는다.

Chapter 2 슬라이드 수치 경계

- ☐ 96.139 GB/s는 bank-scaling sweep의 all-bank 결과로 표시한다.
- ☐ 95.262 GB/s는 request-size factorial의 8 KiB request, 64 KiB batch 결과로 표시한다.

- ☐ 두 수치를 하나의 exact configuration 결과처럼 합치지 않는다.
- ☐ headline은 $\approx 95\text{--}96\text{ GB/s}$ measured read-only transport roof로 쓴다.
- ☐ reader latency, retire wait, bank-pair 및 raw permutation 표는 appendix에 둔다.

완료 조건

각 checkbox는 exact command, source/build checksum, exit status, 반복 통계 및 artifact 경로가 보고서에 추가된 뒤에만 완료 처리한다. Timeout, signal 또는 exit 124/137 결과는 성능 표에 넣지 않고 BOS safety contract에 따라 장치를 격리한다.